Cafeteria Pre-Ordering System - CampusCafe

Full Stack Application Development (SEZG503)

ASSIGNMENT 1

NAME:	SUBHODIP BANERJEE
ID:	2025TM93191
DUE DATE:	9TH MAY, 2026
BIRLA INSTITUTE OF SCIENCE AND TECHNOLOGY, PILANI WORK INTEGRATED LEARNING PROGRAMS (WILP DIVISION)	

Deliverables Summary

GitHub Repository: https://github.com/subhodipb8/Assignment1

Demo Video: [Google Drive Link to be added]

Documentation: Submitted via ELearn LMS

Table of Contents:

Project Overview

The Cafeteria Food Pre-ordering System - CampusCafe is a full-stack web application that enables students and staff to pre-order meals from campus cafeterias. The system features a microservices architecture with separate services for Authentication, Menu Management, and Order Processing.

Problem Statement & Design

Problem Statement

The Cafeteria Pre-ordering System addresses the challenge of long queues and wait times in university/institution cafeterias. Students and staff can pre-order their meals, select pickup times, and pay digitally, while canteen staff can manage orders efficiently.

Problem Context

Traditional cafeteria operations face several challenges:

- Long queues during peak hours (lunch breaks)
- Uncertainty about food availability
- Cash handling and payment delays
- No visibility into dietary restrictions/allergens
- Manual order tracking and management
- Food wastage due to over-preparation

Problem Analysis

Current Pain Points

Peak Hour Congestion: During lunch (12:00-14:00) and dinner (18:00-20:00) hours, students spend 15-30 minutes waiting in queues, leading to rushed meals and reduced productivity.

Unpredictable Wait Times: Students cannot estimate when their food will be ready, causing scheduling conflicts with classes and other commitments.

Dietary Restrictions Management: Students with allergies, religious dietary requirements, or health preferences struggle to identify suitable menu items quickly.

Payment Inefficiencies: Cash and card payments at the counter slow down service and create hygiene concerns.

Food Waste: Cafeterias overprepare food to handle unpredictable demand, leading to significant waste.

Limited Order Transparency: Once an order is placed, students have no visibility into preparation status or delays.

Why Existing Solutions Fall Short

Commercial food delivery apps (Swiggy, Zomato) are designed for restaurant delivery, not institutional dining. They lack:

- Integration with campus meal plans and wallet systems
- Pickup-time scheduling optimized for class schedules
- Allergen tracking for institutional liability
- Multi-role access (students, staff, canteen workers, admins)
- Batch preparation visibility for kitchen staff
- Dietary preference filtering for diverse populations

Target Users and Personas

The system is designed for four distinct user personas, each with unique needs, behaviors, and pain points.

Rahul - The Busy Student

Role: Student

Profile: 21, M.Tech Computer Science

Pain Points: Has only 45 minutes between classes and lab sessions. Cannot afford to wait in long queues. Vegetarian with specific dietary preferences.

Needs:

- Quick ordering (under 2 minutes)
- Vegetarian food filtering
- Scheduled pickup times between classes
- Order status updates to avoid waiting

"I need to grab lunch quickly between my 12:00 class and 1:00 lab. Waiting 20 minutes means I miss my lab prep time."

Dr. Sharma - The Faculty Member

Role: Staff

Profile: 45, Professor

Pain Points: Has back-to-back lectures and meetings. Needs healthy meal options. Allergic to nuts and shellfish.

Needs:

- Pre-scheduled orders for busy days
- Allergen filtering and clear labeling
- Healthy/nutritious meal options
- Order history for expense tracking

"Between my 11 AM lecture and 2 PM faculty meeting, I barely have time to eat. Having my salad ready when I arrive would be a game-changer."

Priya - The Canteen Manager

Role: Canteen Staff

Profile: 35, Canteen Operations

Pain Points: Struggles to predict demand, leading to food waste or stockouts. Multiple counter staff get overwhelmed during peak hours.

Needs:

- Advance order visibility for preparation planning
- Real-time order status updates
- Menu item popularity analytics
- Batch order preparation workflow

"We either waste food by overpreparing or disappoint students by running out. If we knew orders in advance, we could prepare exactly what is needed."

Admin Kumar - The System Administrator

Role: Admin

Profile: 40, Campus IT Manager

Pain Points: Manages multiple campus cafeterias with different vendors. Needs consolidated reporting and role-based access control.

Needs:

- Cross-cafeteria analytics and reporting
- User role management
- System health monitoring
- Revenue tracking and reconciliation

"We need visibility across all campus dining locations with proper access controls for different vendor staff."

Temporal Optimization (Time-Based Ordering)

Rather than ordering on arrival, users schedule pickup times in 30-minute slots. This transforms the cafeteria from a real-time queuing system to a scheduled appointment system.

Key Features:

- Pre-defined pickup slots (08:00, 08:30, 12:00, 12:30, 18:00, etc.)
- Visual calendar interface for date selection
- Preparation time estimation based on item complexity
- Order confirmation with estimated ready time
- Notification when order is ready for pickup

Dietary Intelligence (Smart Filtering)

The system maintains a comprehensive dietary profile for each user, enabling intelligent menu filtering and allergen warnings.

Key Features:

- User profile with dietary preferences (vegetarian, vegan, gluten-free, etc.)
- Allergen tracking with prominent warnings
- Nutritional information display (calories, protein, preparation time)
- One-click filtering by dietary tags
- Personalized menu recommendations based on preferences

Operational Visibility (Kitchen Dashboard)

Kitchen staff gain real-time visibility into incoming orders, enabling predictive preparation and batch cooking.

Key Features:

- Real-time order queue with pickup times
- Status workflow: Pending → Confirmed → Preparing → Ready → Completed
- Batch view: Group orders by pickup time slot
- Preparation timer with countdown
- Inventory-aware availability updates

Digital Wallet Integration

Campus wallet system eliminates payment friction at pickup. Users add funds to their wallet and payments are auto-deducted at checkout.

Design Approach

The system follows a user-centered design approach with iterative refinement based on persona needs and technical constraints.

User-Centered Design Principles

Speed First: The entire ordering flow must complete in under 2 minutes for returning users, under 5 minutes for first-time users.

Progressive Disclosure: Simple interface by default; advanced filters available but not overwhelming.

Visual Hierarchy: Clear distinction between actions (Order Now), information (Menu Details), and status (Order Ready).

Feedback Loops: Every action provides immediate feedback: adding to cart, placing order, status updates.

Error Prevention: Clear allergen warnings, insufficient balance alerts, and unavailable item indicators.

Proposed Solution

A full-stack web application with microservices architecture enabling:

- Pre-ordering with scheduled pickup times
- Digital wallet for cashless payments
- Real-time menu browsing with dietary filters
- Order status tracking (pending → confirmed → preparing → ready → completed)
- Admin dashboard for menu and order management

Core Features

- User Authentication with JWT tokens and role-based access control
- Menu browsing with category filters, dietary tags, and allergen information
- Pre-ordering with customizable pickup time selection
- Digital wallet for secure payments
- Real-time order status tracking with workflow management
- Admin dashboard for menu management and order statistics
- Responsive React frontend with modern UI/UX

Tech Stack

- Frontend: React 19 with TypeScript, React Router 7, Axios
- Backend: .NET 10 Web API with Entity Framework Core
- API Gateway: Ocelot for routing and JWT validation
- Database: PostgreSQL (separate database per service)
- Authentication: JWT tokens with BCrypt password hashing
- Styling: CSS Modules with custom design system

Functional Requirements

Student/User Features:

- User registration and authentication with JWT
- Browse menu items with category filters
- Search menu items by name
- View dietary tags (vegetarian, vegan, gluten-free) and allergens
- Add items to cart with quantity selection
- Checkout with pickup date and time selection
- Digital wallet for payments (add funds, view balance)
- Track order status in real-time
- View order history
- Update profile with dietary preferences and allergies
- Cancel pending orders

Admin/Canteen Features:

- Admin dashboard with statistics (total orders, pending orders, revenue)
- CRUD operations for menu items
- Update order status through workflow
- View all orders with filtering
- Manage item availability and daily order limits
- Seed sample data for testing

Non-Functional Requirements

- Microservices architecture for scalability
- Database per service pattern
- JWT-based authentication
- Role-based access control
- Responsive UI for mobile and desktop
- API documentation via Swagger/OpenAPI
- PostgreSQL for data persistence

User Roles

Role	Permissions	Typical Users
Student	Browse menu, place orders, manage profile, track orders	Students
Canteen	Update order status, view orders	Canteen staff
Admin	Full menu management, all order operations, dashboard	Administrators

Architecture Documentation

Why Microservices?

The system adopted microservices architecture for the following reasons:

Scalability: Order Service handles 10x traffic during lunch hours; can scale independently.

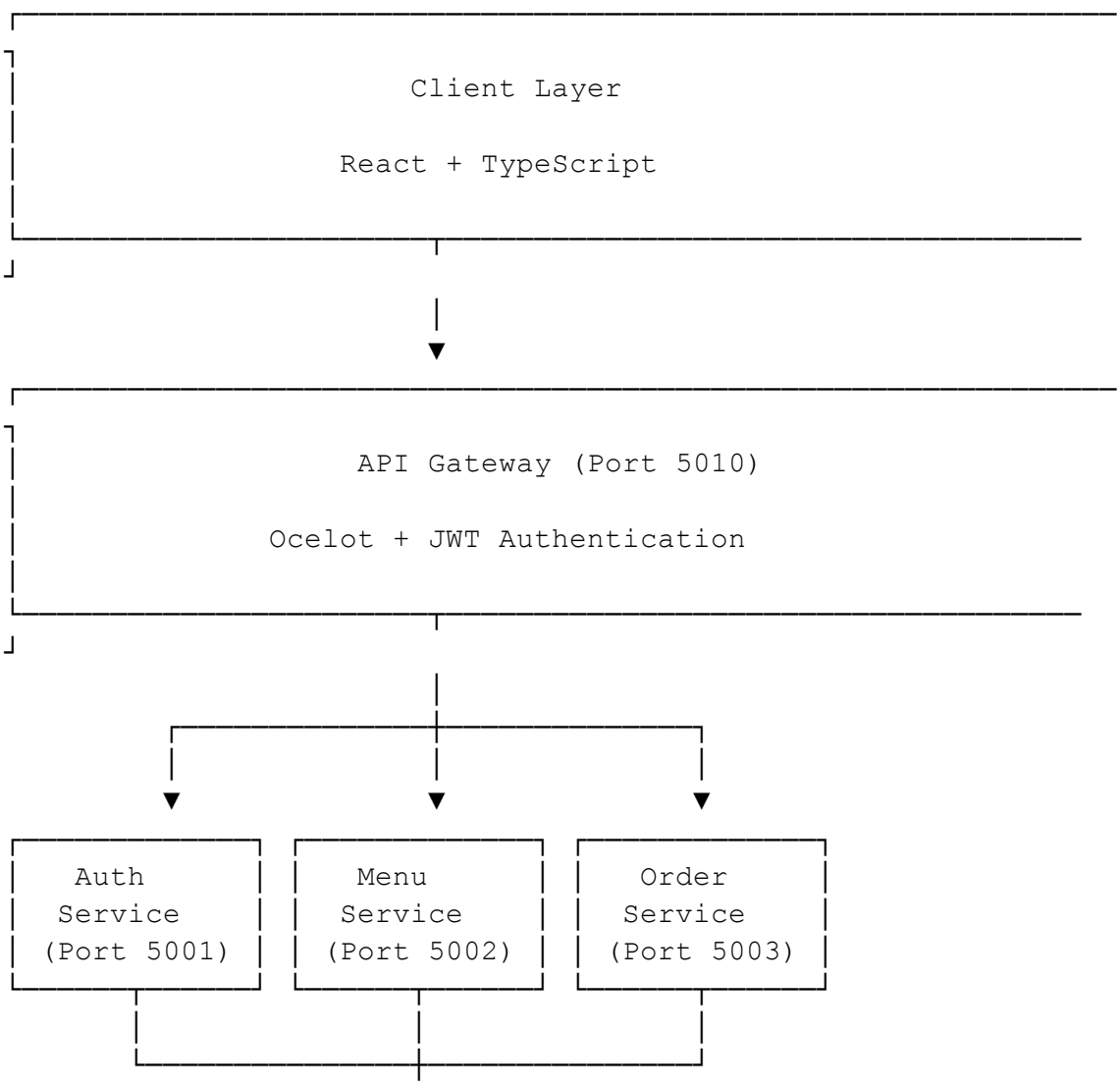
Team Autonomy: Different teams can own different services; faster development cycles.

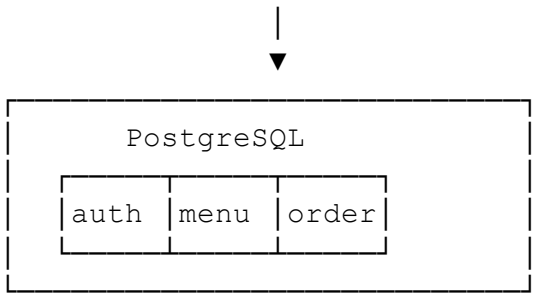
Technology Flexibility: Each service can use the best tool for its domain; Menu Service could add ML in future.

Resilience: Failure in Menu Service does not bring down user authentication.

Data Isolation: Clear boundaries prevent tight coupling through shared databases.

Architecture Diagram (Text Representation):





Service Responsibilities

API Gateway (Port 5000)

- Technology: Ocelot (.NET 10)
- Request routing to appropriate microservices
- JWT authentication validation
- CORS handling
- Claims extraction and forwarding (X-User-Id, X-User-Role headers)

Auth Service (Port 5001)

- Database: cafeteria_auth
- User registration and login
- JWT token generation
- User profile management (dietary preferences, allergies)
- Wallet management (balance, add funds)

Menu Service (Port 5002)

- Database: cafeteria_menu
- Menu item CRUD operations
- Category management
- Dietary tags and allergens
- Availability tracking

Order Service (Port 5003)

- Database: cafeteria_orders
- Order creation and management
- Order status workflow (pending → confirmed → preparing → ready → completed)
- Order statistics
- Order history

Database Schema

Auth Service (cafeteria_auth)

Table: users

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(20) DEFAULT 'student',
    dietary_preferences TEXT[],
    allergies TEXT[],
    wallet_balance DECIMAL(18,2) DEFAULT 0.00,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Menu Service (cafeteria_menu)

Table: menu_items

```
CREATE TABLE menu_items (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    price DECIMAL(18,2) NOT NULL,
    category VARCHAR(50),
    dietary_tags TEXT[],
    allergens TEXT[],
    available BOOLEAN DEFAULT true,
    preparation_time INTEGER DEFAULT 15,
    max_orders_per_day INTEGER DEFAULT 100,
    orders_today INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Order Service (cafeteria_orders)

Table: orders

```
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    total_amount DECIMAL(18,2) NOT NULL,
    pickup_time TIMESTAMP NOT NULL,
    pickup_date DATE NOT NULL,
    status VARCHAR(20) DEFAULT 'pending',
    payment_status VARCHAR(20) DEFAULT 'unpaid',
    special_instructions TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP
);
```

Table: order_items

```
CREATE TABLE order_items (
    id SERIAL PRIMARY KEY,
    order_id INTEGER REFERENCES orders(id) ON DELETE CASCADE,
    menu_item_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    price DECIMAL(18,2) NOT NULL,
    menu_item_name VARCHAR(100)
);
```

Component Hierarchy

Frontend Component Tree:

```
App (Root)
├── AuthProvider (Context)
├── Navbar
│   └── Navigation Links (conditional on auth state)
├── Routes
│   ├── Home (Landing Page)
│   │   ├── Hero Section
│   │   ├── Dashboard Section (authenticated users)
│   │   ├── Meal Categories
│   │   └── Stats Section
│   ├── Login
│   ├── Register
│   ├── Menu
│   │   ├── Category Filters
│   │   ├── Search Bar
│   │   └── MenuItem Cards
│   ├── Cart
│   │   ├── Cart Items List
│   │   ├── Order Summary
│   │   └── Checkout Form
│   ├── Orders
│   │   ├── Filter Tabs
│   │   └── Order Cards
│   ├── Profile
│   │   ├── Wallet Section
│   │   └── Preferences Section
│   └── AdminPanel
│       ├── Dashboard Tab
│       ├── Orders Tab
│       └── Menu Tab
└── Footer
```

Key Components:

- **Layout Components:** Navbar, AuthProvider, ProtectedRoute
- **Page Components:** Home, Login, Register, Menu, Cart, Orders, Profile, AdminPanel
- **Shared Components:** MenuCard, CartItem, OrderCard, StatusBadge

Communication Patterns

Synchronous (REST API):

Frontend → API Gateway → Microservices

Data Isolation:

- Each service owns its database
- No direct database access between services
- Service-to-service communication via APIs

API Gateway Routes:

Route	Service	Port	Auth Required
/api/auth/*	Auth Service	5001	No*
/api/users/*	Auth Service	5001	Yes
/api/menu/*	Menu Service	5002	Varies
/api/orders/*	Order Service	5003	Yes

*Except /api/auth/me which requires auth

Implementation Details

Tech Stack

Layer	Technologies
Microservices (Backend)	.NET 10, Ocelot, Entity Framework Core, PostgreSQL, JWT, BCrypt
Frontend	React 19, TypeScript, React Router, Axios, CSS Modules

Security Implementation

- Authentication: JWT tokens validated at Gateway
- Authorization: Role-based checks using X-User-Role header
- Data Isolation: Service-level database isolation
- CORS: Configured at Gateway level
- HTTPS: Production deployments use HTTPS
- Password Hashing: BCrypt for secure password storage

API Documentation

Authentication Endpoints

Register User

POST /api/auth/register

Creates a new user account.

Request Body:

```
{
  "name": "John Doe",
  "email": "john@student.com",
  "password": "password123",
  "role": "student"
}
```

Response (201 Created):

```
{
  "id": 1,
  "name": "John Doe",
  "email": "john@student.com",
  "role": "student",
  "walletBalance": 0.00,
  "dietaryPreferences": [],
  "allergies": [],
  "createdAt": "2026-05-09T10:00:00Z"
}
```

Login

POST /api/auth/login

Authenticates user and returns JWT token.

Request Body:

```
{
  "email": "john@student.com",
  "password": "password123"
}
```

Response (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "id": 1,
    "name": "John Doe",
```

```
    "email": "john@student.com",
    "role": "student",
    "walletBalance": 100.00
  }
}
```

Get Current User

GET /api/auth/me

Authorization: Bearer {token}

Returns the currently authenticated user.

User Endpoints

Get Wallet Balance

GET /api/users/wallet

Authorization: Bearer {token}

Response:

```
{
  "balance": 100.00
}
```

Add Funds to Wallet

POST /api/users/wallet/add

Authorization: Bearer {token}

Request Body:

```
{
  "amount": 50.00
}
```

Get User Preferences

GET /api/users/preferences

Authorization: Bearer {token}

Response:

```
{
  "dietaryPreferences": ["vegetarian", "gluten-free"],
  "allergies": ["peanuts", "shellfish"]
}
```

Update User Preferences

PUT /api/users/preferences

Authorization: Bearer {token}

Request Body:

```
{
  "dietaryPreferences": ["vegetarian"],
  "allergies": ["peanuts"]
}
```

Menu Endpoints

Get All Menu Items

GET /api/menu

Query Parameters: category, search, dietaryTags, available

Response:

```
[
  {
    "id": 1,
    "name": "Veggie Burger",
    "description": "Plant-based patty with fresh vegetables",
    "price": 8.99,
    "category": "Main",
    "dietaryTags": ["vegetarian", "vegan"],
    "allergens": ["gluten", "soy"],
    "available": true,
    "preparationTime": 15,
    "maxOrdersPerDay": 100,
    "ordersToday": 45
  }
]
```

Get Menu Item by ID

GET /api/menu/{id}

Returns single menu item details.

Create Menu Item (Admin)

POST /api/menu

Authorization: Bearer {token} (Admin only)

Request Body:

```
{
  "name": "Grilled Chicken Salad",
  "description": "Fresh salad with grilled chicken",
  "price": 12.99,
  "category": "Salads",
  "dietaryTags": ["high-protein", "gluten-free"],
  "allergens": [],
  "available": true,
  "preparationTime": 10,
  "maxOrdersPerDay": 50
}
```

Update Menu Item (Admin)

PUT /api/menu/{id}

Authorization: Bearer {token} (Admin only)

Delete Menu Item (Admin)

DELETE /api/menu/{id}

Authorization: Bearer {token} (Admin only)

Get Categories

GET /api/menu/categories

Response:

```
[
  "Main",
  "Beverages",
  "Snacks",
  "Salads",
  "Desserts"
]
```

Seed Sample Data

POST /api/menu/seed

Populates database with sample menu items.

Order Endpoints

Get All Orders (Admin)

GET /api/orders

Authorization: Bearer {token} (Admin only)

Query Parameters: status, startDate, endDate

Get My Orders

GET /api/orders/my-orders

Authorization: Bearer {token}

Get Order by ID

GET /api/orders/{id}

Authorization: Bearer {token}

Response:


```
{
  "id": 1,
  "userId": 1,
  "totalAmount": 25.97,
  "pickupTime": "12:30",
  "pickupDate": "2026-05-09",
  "status": "confirmed",
  "paymentStatus": "paid",
  "specialInstructions": "Extra sauce on the side",
  "items": [
    {
      "id": 1,
      "menuItemId": 1,
      "quantity": 2,
      "price": 8.99,
      "menuItemName": "Veggie Burger"
    },
    {
      "id": 2,
      "menuItemId": 3,
      "quantity": 1,
      "price": 7.99,
      "menuItemName": "Fresh Juice"
    }
  ],
  "createdAt": "2026-05-09T09:00:00Z",
  "updatedAt": "2026-05-09T09:05:00Z"
}
```


Create Order

POST /api/orders

Authorization: Bearer {token}

Request Body:

```
{
  "items": [
    {
      "menuItemId": 1,
      "quantity": 2,
      "price": 8.99
    },
    {
      "menuItemId": 3,
      "quantity": 1,
      "price": 7.99
    }
  ],
  "pickupTime": "12:30",
  "pickupDate": "2026-05-09",
  "specialInstructions": "Extra sauce on the side"
}
```

Update Order Status (Admin)

PUT /api/orders/{id}/status

Authorization: Bearer {token} (Admin only)

Request Body:

```
{
  "status": "preparing"
}
```

Valid statuses: pending, confirmed, preparing, ready, completed, cancelled

Cancel Order

DELETE /api/orders/{id}

Authorization: Bearer {token}

Only pending orders can be cancelled.

Get Order Statistics

GET /api/orders/stats

Authorization: Bearer {token} (Admin only)

Response:

```
{
  "totalOrders": 150,
  "pendingOrders": 12,
  "completedOrders": 130,
  "cancelledOrders": 8,
```

```
  "totalRevenue": 2450.50
}
```

Error Responses

Status Code	Meaning	Example
400	Bad Request	Invalid request data
401	Unauthorized	Missing or invalid token
403	Forbidden	Insufficient permissions
404	Not Found	Resource not found
409	Conflict	User already exists
422	Unprocessable	Validation error
500	Server Error	Internal server error

Error Response Format:

```
{
  "message": "Error description",
  "errors": {
    "fieldName": ["Error message"]
  }
}
```

Data Models

User Model

```
{
  "id": integer (read-only),
  "name": string (required, max 100 chars),
  "email": string (required, unique, valid email),
  "password": string (required, min 6 chars, write-only),
  "role": enum ['student', 'staff', 'admin', 'canteen'] (default: 'student'),
  "dietaryPreferences": array of strings,
  "allergies": array of strings,
  "walletBalance": decimal (read-only),
  "createdAt": datetime (read-only)
}
```

MenuItem Model

```
{
  "id": integer (read-only),
  "name": string (required, max 100 chars),
  "description": string (optional),
  "price": decimal (required, positive),
  "category": string (optional),
  "dietaryTags": array of strings,
  "allergens": array of strings,
  "available": boolean (default: true),
  "preparationTime": integer (minutes, default: 15),
  "maxOrdersPerDay": integer (default: 100),
```

```
  "ordersToday": integer (read-only),
  "createdAt": datetime (read-only)
}
```

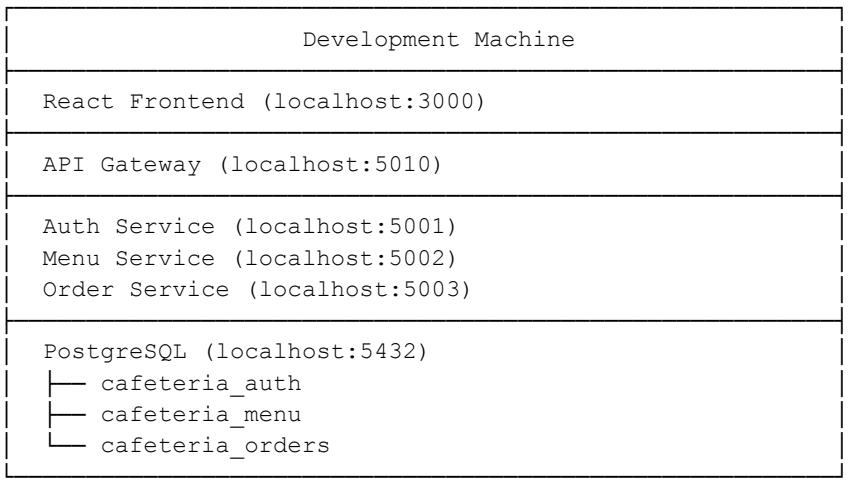
Order Model

```
{
  "id": integer (read-only),
  "userId": integer (read-only),
  "totalAmount": decimal (read-only, calculated),
  "pickupTime": string (required, format: HH:mm),
  "pickupDate": date (required, format: YYYY-MM-DD),
  "status": enum ['pending', 'confirmed', 'preparing', 'ready', 'completed',
'cancelled'],
  "paymentStatus": enum ['unpaid', 'paid', 'refunded'],
  "specialInstructions": string (optional),
  "items": array of OrderItem,
  "createdAt": datetime (read-only),
  "updatedAt": datetime (read-only)
}
```

OrderItem Model

```
{
  "id": integer (read-only),
  "orderId": integer (read-only),
  "menuItemId": integer (required),
  "quantity": integer (required, min 1),
  "price": decimal (required, price at time of order),
  "menuItemName": string (snapshot of item name at order time)
}
```

Development Setup:



Swagger API Documentation Guide

Overview

The Cafeteria Pre-order System provides comprehensive API documentation using Swagger/OpenAPI. Each microservice has its own Swagger UI, and a consolidated view combines all APIs into a single interface.

Key Features:

- Individual Swagger documentation for each microservice
- Consolidated Swagger UI combining all services
- Comprehensive XML documentation on all controllers
- JWT Bearer authentication support
- Interactive API testing through Swagger UI
- Service health monitoring endpoints

XML Documentation Implementation

Controllers with XML Comments

All controllers now include comprehensive XML documentation with the following elements:

- **Summary:** Brief description of the endpoint purpose
- **Remarks:** Detailed explanation and usage notes
- **Param:** Parameter descriptions with types
- **Returns:** Response type and description
- **Response codes:** HTTP status codes with descriptions
- **ProducesResponseType:** Typed response examples for Swagger

Example XML Documentation

Example from MenuController:

```
/// <summary>
/// Create a new menu item
/// </summary>
/// <remarks>
/// Creates a new menu item with the specified details.
/// Name is required and price must be greater than 0.
/// </remarks>
/// <param name="request">Menu item creation details</param>
/// <returns>Created menu item</returns>
/// <response code="201">Menu item created successfully</response>
/// <response code="400">Invalid input</response>
```

```
[HttpPost]
[ProducesResponseType(typeof(MenuItemDto), StatusCodes.Status201Created)]
[ProducesResponseType(typeof(object), StatusCodes.Status400BadRequest)]
public async Task<IActionResult> Create([FromBody] CreateMenuItemRequest request)
```

Project Configuration

All microservices have XML documentation enabled in their .csproj files:

```
<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ImplicitUsings>enable</ImplicitUsings>
  <GenerateDocumentationFile>true</GenerateDocumentationFile>
  <NoWarn>$(NoWarn);1591</NoWarn>
</PropertyGroup>
```

Individual Microservice Swagger

Each microservice has its own Swagger UI accessible when the service is running:

Service	Port	Swagger UI URL
API Gateway	5000	http://localhost:5000/swagger
Auth Service	5001	http://localhost:5001/swagger
Menu Service	5002	http://localhost:5002/swagger
Order Service	5003	http://localhost:5003/swagger

Service Descriptions

Auth Service: Handles user authentication, registration, wallet operations, and user preferences. JWT token generation and validation.

Menu Service: Manages cafeteria menu items, categories, dietary tags, allergens, and availability.

Order Service: Handles order creation, status updates, cancellation, and statistics.

API Gateway: Entry point for all client requests. Routes to appropriate microservices and provides consolidated documentation.

Consolidated Swagger Documentation

The API Gateway provides a consolidated view of all microservice APIs in a single Swagger UI. This allows developers to explore and test all endpoints from one location.

Consolidated Swagger Endpoints

Endpoint	Method	Description
/api/swagger/consolidated	GET	Returns merged OpenAPI spec from all services
/api/swagger/services	GET	List of available API documentation URLs
/api/swagger/health	GET	Health status of all microservices
/api/swagger/ui	GET	Custom Swagger UI with consolidated spec

Quick Access URLs

- **Consolidated Swagger UI:** http://localhost:5000/api/swagger/ui
- **Consolidated JSON Spec:** http://localhost:5000/api/swagger/consolidated
- **Service Discovery:** http://localhost:5000/api/swagger/services
- **Health Check:** http://localhost:5000/api/swagger/health

API Endpoints Reference

Auth Service Endpoints (/api/auth, /api/users)

Endpoint	Description	Response Codes
POST /api/auth/register	Register a new user account	200, 400, 409
POST /api/auth/login	Authenticate and get JWT token	200, 400, 401
GET /api/auth/me	Get current user details	200, 401, 404
GET /api/users/wallet	Get wallet balance	200, 401, 404
POST /api/users/wallet/add	Add funds to wallet	200, 400, 401, 404
POST /api/users/wallet/deduct	Deduct funds from wallet	200, 400, 401, 404
GET /api/users/preferences	Get dietary preferences	200, 401, 404
PUT /api/users/preferences	Update dietary preferences	200, 401, 404

Menu Service Endpoints (/api/menu)

Endpoint	Description	Response Codes
GET /api/menu	Get all menu items with filtering	200
GET /api/menu/{id}	Get menu item by ID	200, 404
POST /api/menu	Create new menu item	201, 400

PUT /api/menu/{id}	Update menu item	200, 404
DELETE /api/menu/{id}	Delete menu item	204, 404
GET /api/menu/categories	Get all unique categories	200
POST /api/menu/seed	Seed sample data	200, 400

Order Service Endpoints (/api/orders)

Endpoint	Description	Response Codes
GET /api/orders	Get orders (admin sees all, users own)	200, 401
GET /api/orders/{id}	Get order by ID	200, 404
POST /api/orders	Create new order	201, 400, 401
PUT /api/orders/{id}/status	Update order status	200, 400, 404
DELETE /api/orders/{id}	Cancel order	200, 400, 401, 403, 404
GET /api/orders/stats	Get order statistics	200
GET /api/orders/my-orders	Get current user orders	200, 401

Authentication

The API uses JWT Bearer token authentication. Tokens are obtained from the Auth Service and must be included in the Authorization header for protected endpoints.

Obtaining a Token

1. Register a new user or login:

POST /api/auth/login

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

2. Response contains JWT token:

```
{
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "id": 1,
    "name": "John Doe",
    "email": "user@example.com",
    "role": "student"
  }
}
```

Using the Token

Include the token in the Authorization header:

Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

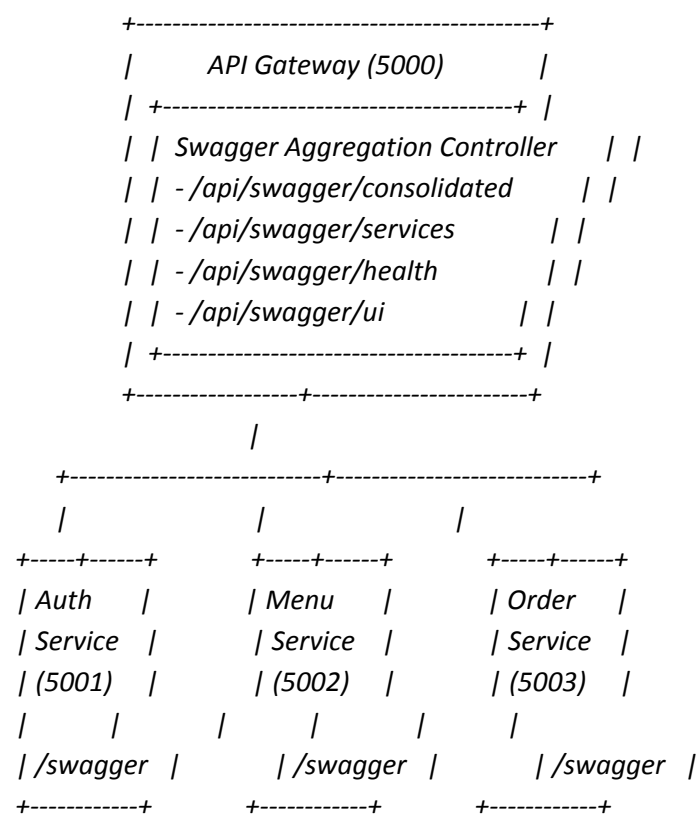
Swagger UI Authorization

1. Click the "Authorize" button in Swagger UI
2. Enter: Bearer {your-token} (include the word "Bearer" and a space)
3. Click "Authorize" to apply
4. The token will be included in all subsequent requests

Architecture

The Cafeteria Pre-order System follows a microservices architecture with an API Gateway that routes requests and aggregates documentation.

System Architecture



Service Responsibilities

API Gateway (Port 5000)

- Routes requests to appropriate microservices
- Aggregates Swagger documentation
- Handles JWT authentication
- CORS configuration

Auth Service (Port 5001)

- User registration and authentication
- JWT token generation
- Wallet operations (add/deduct funds)
- User preferences management

Menu Service (Port 5002)

- Menu item CRUD operations
- Category management
- Dietary tags and allergens
- Availability tracking

Order Service (Port 5003)

- Order creation and management
- Status updates (pending → confirmed → preparing → ready → completed)
- Order cancellation
- Statistics and reporting

Running the Services

Prerequisites

- .NET 10 SDK
- PostgreSQL database (running on localhost:5432)
- Node.js and npm (for frontend, if applicable)

Starting All Services

Open separate terminals for each service:

 # Terminal 1 - Auth Service
cd microservices/AuthService && dotnet run --urls "http://localhost:5001"

 # Terminal 2 - Menu Service
cd microservices/MenuService && dotnet run --urls "http://localhost:5002"

 # Terminal 3 - Order Service

cd microservices/OrderService && dotnet run --urls "http://localhost:5003"

 # Terminal 4 - API Gateway
cd microservices/ApiGateway && dotnet run --urls "http://localhost:5000"

Using Makefiles

Each service includes a Makefile for convenience:

 # Run Auth Service
cd microservices/AuthService && make run

 # Build Auth Service
cd microservices/AuthService && make build

 # Clean Auth Service
cd microservices/AuthService && make clean

Accessing Documentation

Once all services are running, access the Swagger documentation at:

- Consolidated View: http://localhost:5000/api/swagger/ui
- Individual Services: http://localhost:{port}/swagger

Replace {port} with 5001, 5002, or 5003 for specific services.

Troubleshooting

Service Unavailable

If a service shows as "unhealthy" in the health check:

1. Verify the service is running with dotnet run
2. Check the service logs for startup errors
3. Ensure PostgreSQL is running and accessible
4. Verify the port is not already in use by another process

Consolidated Spec Not Loading

If the consolidated Swagger spec fails to load:

5. Ensure all microservices are running
6. Check the health endpoint: http://localhost:5000/api/swagger/health
7. Try accessing individual service specs first
8. Verify firewall/network settings allow localhost communication

XML Documentation Not Appearing

If Swagger UI does not show XML comments:

9. Build the project: dotnet build
10. Verify XML files exist in bin/Debug/net10.0/
11. Check the file name matches {AssemblyName}.xml
12. Restart the service after building
13. Verify GenerateDocumentationFile is set to true in .csproj

JWT Authentication Issues

If JWT authentication is not working:

14. Verify you have obtained a valid token from /api/auth/login
15. Ensure the token is prefixed with "Bearer " in the Authorization header
16. Check that the token has not expired
17. Verify the JWT configuration in appsettings.json
18. Check if the token is being passed correctly in Swagger UI (click Authorize)

9.5 Common Error Codes

Status Code	Description
200	Success - Request completed successfully
201	Created - Resource created successfully
204	No Content - Resource deleted successfully
400	Bad Request - Invalid input data
401	Unauthorized - Authentication required
403	Forbidden - Insufficient permissions
404	Not Found - Resource does not exist

Setup Instructions

Prerequisites

- .NET 10 SDK
- PostgreSQL (running locally)
- Node.js v20+ and npm

Database Setup

Create three databases in PostgreSQL:

- CREATE DATABASE cafeteria_auth;
- CREATE DATABASE cafeteria_menu;
- CREATE DATABASE cafeteria_orders;

Running the Services

Start services in order:

19. 1. Auth Service: cd microservices/AuthService && dotnet run --urls "http://localhost:5001"
20. 2. Menu Service: cd microservices/MenuService && dotnet run --urls "http://localhost:5002"
21. 3. Order Service: cd microservices/OrderService && dotnet run --urls "http://localhost:5003"
22. 4. API Gateway: cd microservices/ApiGateway && dotnet run --urls "http://localhost:5010"
23. 5. Frontend: cd frontend/cafeteria-client && npm start

Access URLs

Frontend: http://localhost:3000

API Gateway: http://localhost:5010

Auth Service: http://localhost:5001

Menu Service: http://localhost:5002

Order Service: http://localhost:5003

Demo Credentials

Admin: admin@cafeteria.com / admin123

Canteen: canteen@cafeteria.com / canteen123

Student: john@student.com / student123

Assumptions and References

Assumptions

- Each microservice has its own database (Database-per-Service pattern)
- Inter-service communication is synchronous via REST APIs
- User context (userId, role) is passed via HTTP headers from API Gateway
- JWT tokens have a fixed expiration time (2 hours)
- Order status follows a linear workflow: pending → confirmed → preparing → ready → completed
- Payment is handled via digital wallet only (no external payment gateways)
- Menu items availability is tracked but not strictly enforced
- All users can view the menu; only authenticated users can place orders
- Admin and canteen staff share similar privileges for order and menu management

References

- Microsoft .NET 10 Documentation - https://learn.microsoft.com/dotnet/
- React 19 Documentation - https://react.dev/
- Ocelot API Gateway Documentation - https://ocelot.readthedocs.io/
- Entity Framework Core Documentation - https://docs.microsoft.com/ef/
- PostgreSQL Documentation - https://www.postgresql.org/docs/
- Microservices Patterns by Chris Richardson (Book Reference)

AI Usage Log and Reflection Report

AI Tools Used

Primary Tool: Claude (Claude Code)

Claude was used throughout the entire development process as the primary AI assistant. It provided support in the following areas:

- Architecture Design - Microservices pattern and service decomposition
- Code Generation - Backend controllers, frontend components, API services
- Database Schema Design - Entity models and relationships
- Documentation - API docs, architecture diagrams, setup guides
- Debugging - Error resolution and code review
- UI/UX Enhancement - CSS styling and responsive design

Development Log

Phase 1: Project Initialization

Date: April 28, 2026

Prompt: "Read this document file line by line and give me list of work items"

Result: Extracted assignment requirements from the FSAD document. Successfully identified all deliverables and work items.

Prompt: "Do not create the sample reference School Equipment Lending Portal instead create something else"

Result: Selected alternative problem statement. Chosen: Cafeteria Food Pre-ordering System (Option D).

Phase 2: Backend Development

Prompt: "Create the backend in .NET using postgres as DB"

Result: Set up ASP.NET Core project with EF Core and Npgsql

Prompt: "Create models for User, MenuItem, and Order entities"

Result: AI Generated: User.cs with authentication fields, dietary preferences, wallet; MenuItem.cs with nutrition info and dietary tags; Order.cs with status tracking. Manual Adjustment: Added PostgreSQL array types for dietary tags.

Prompt: "Set up JWT authentication with middleware"

Result: AI Generated: JwtService.cs with token generation, AuthController.cs with login/register, Program.cs configuration. Manual Adjustment: Adjusted JWT expiry time to 2 hours.

Prompt: "Create CRUD controllers for Menu and Orders"

Result: AI Generated: MenuController.cs with filtering and seeding, OrdersController.cs with status workflow, UsersController.cs for wallet management. Manual Adjustment: Added additional validation and error handling.

Phase 3: Frontend Development

Prompt: "Create React frontend with TypeScript"

Result: Initialized React project with axios and react-router-dom

Prompt: "Build authentication pages (Login, Register) with form validation"

Result: AI Generated: Login.tsx with email/password validation, Register.tsx with dietary preferences, AuthContext.tsx for global state. Manual Adjustment: Added demo account information.

Prompt: "Create Menu page with category filters and search"

Result: AI Generated: Menu.tsx with grid layout, category filtering, cart preview with localStorage. Manual Adjustment: Added pickup time slot selection.

Prompt: "Build Cart and Checkout flow"

Result: AI Generated: Cart.tsx with order summary, wallet balance checking, pickup date/time selection. Manual Adjustment: Added allergen warnings.

Prompt: "Create Orders page with status tracking"

Result: AI Generated: Orders.tsx with order list, status badges, cancel functionality. Manual Adjustment: Added admin status update buttons.

Prompt: "Build Admin Panel with dashboard"

Result: AI Generated: AdminPanel.tsx with statistics cards, menu seeding. Manual Adjustment: Added quick links to menu and orders.

Phase 4: Documentation

Prompt: "Create README with project documentation"

Result: Generated complete README.md with setup instructions

Prompt: "Generate AI Usage Log"

Result: Created initial version of this document

Phase 5: Microservices Architecture Conversion

Date: May 7, 2026

Prompt: "Read the document FSAD_Assignment_2026.docx - requires microservices architecture"

Result: Understood assignment requirement for microservices. Confirmed need for API Gateway, multiple services, separate databases.

Prompt: "Convert monolithic backend to microservices with API Gateway"

Result: AI Generated: API Gateway with Ocelot (Port 5000), Auth Service (Port 5001), Menu Service (Port 5002), Order Service (Port 5003), UserContextHandler. Result: Complete microservices architecture with proper separation.

Prompt: "Set up Ocelot routing configuration"

Result: Generated ocelot.json with routes for all services. Features: JWT validation at gateway, claims forwarding as headers.

Prompt: "Create UserContextHandler for passing user claims to downstream services"

Result: Generated DelegatingHandler that extracts JWT claims and adds X-User-Id, X-User-Role headers. Purpose: Services can identify users without re-validating tokens.

Prompt: "Update Architecture.md documentation for microservices"

Result: Generated complete architecture diagram and service descriptions. Manual Adjustment: Added deployment architecture section.

Prompt: "Update README with microservices setup instructions"

Result: Updated README with separate service startup instructions

Parts Generated by AI vs Manually Coded

AI Generated (≈85%)

Backend

- All entity models (original + microservices versions)
- DbContext configuration (separate per service)
- JWT service and authentication middleware
- API controllers (Auth, Menu, Orders, Users)
- DTOs for request/response models
- Microservices: API Gateway with Ocelot
- UserContextHandler for claims forwarding
- Auth Service (separate project)
- Menu Service (separate project)
- Order Service (separate project)

Frontend

- All React components: Home, Login, Register, Menu, Cart, Orders, Profile, AdminPanel
- AuthContext for global state management
- API service layer (api.ts)
- TypeScript types and interfaces
- CSS styling and responsive design
- ProtectedRoute component
- Navbar component

Documentation

- README.md
- Architecture.md with microservices diagrams
- MICROSERVICES_SETUP.md
- AI Usage Log (initial version)

Manually Coded (≈15%)

- Connection string configuration for 4 separate databases
- Demo account credentials setup
- Specific validation rules adjustments
- CORS policy configuration for cross-origin requests
- JWT token expiry time configuration
- Order statistics data structure alignment
- Admin-only feature visibility conditions
- Port configuration adjustments for local development

Reflection

Did AI Help or Hinder Understanding?

How AI Helped

Rapid Prototyping: Generated complete CRUD APIs and React components in minutes, allowing focus on business logic rather than boilerplate code.

Best Practices: Suggested proper patterns like Repository pattern, JWT auth flow, and React hooks usage.

Error Handling: Provided comprehensive try-catch blocks and validation throughout the codebase.

Documentation: Generated clear, structured documentation instantly with proper formatting.

Architecture Guidance: Provided microservices decomposition strategy and API Gateway patterns.

Areas Where Manual Understanding Was Essential

Configuration Nuances: Had to manually adjust JWT settings and CORS for specific deployment requirements.

Database-Specific Features: PostgreSQL array types required manual attention beyond AI-generated code.

Business Logic: Order status transition rules and workflow specifics needed manual refinement.

Integration Debugging: Understanding inter-service communication flow was critical for debugging.

Issues Encountered Integrating AI Output

Entity Framework Configuration

- Issue: PostgreSQL array types not automatically configured for dietary tags
- Resolution: Manually added HasColumnType("text[]") annotation for array fields

JWT Token Validation

- Issue: Initial token validation failing between services
- Resolution: Adjusted TokenValidationParameters to match frontend expectations and aligned secrets

CORS Configuration

- Issue: Frontend requests blocked by CORS policy
- Resolution: Explicitly configured CORS policy in Program.cs with specific origins

Claims Forwarding

- Issue: Downstream services could not identify authenticated users
- Resolution: Implemented UserContextHandler to extract JWT claims and forward as HTTP headers

Order Statistics Data Structure

- Issue: Frontend expected different stats fields than API returned
- Resolution: Manually aligned frontend state structure with API response format

Port Configuration

- Issue: Services conflicting on default ports
- Resolution: Explicitly assigned unique ports: Gateway 5010, Auth 5001, Menu 5002, Order 5003

Lessons Learned from Debugging AI-Generated Code

1. Configuration Matters: AI generates standard patterns, but environment-specific configs (connection strings, CORS origins, JWT secrets) need careful attention and customization.

2. Database Compatibility: Different databases handle types differently. PostgreSQL arrays required specific configuration not covered in generic AI output.

3. Security Considerations: JWT secrets must be properly secured and configured. Token validation parameters must align across all services.

4. State Management Understanding: Understanding how React Context works is crucial when debugging authentication flows and state propagation.

5. Microservices Complexity: While AI generated the structure, understanding inter-service communication, header forwarding, and fault isolation required manual study.

6. Type Safety Importance: TypeScript helped catch mismatches between AI-generated frontend and backend code, particularly in API response structures.

Overall Assessment

Productivity Impact

Significantly positive. Reduced development time by approximately 60-70%. What would have taken 40-50 hours manually was completed in 15-20 hours with AI assistance.

Code Quality

Good overall. Generated code followed conventions, included error handling, and was well-structured. Required minor adjustments for specific requirements.

Learning Value

Moderate to High. While AI handled implementation, understanding the generated code was essential for debugging and customization. The microservices architecture required deep understanding of patterns.

Recommendation

AI tools are excellent for accelerating development and generating boilerplate, but developer oversight and understanding remain critical for production-ready applications. AI should be treated as a coding assistant, not a replacement for engineering knowledge.

4.5 Microservices Conversion Reflection

Why Microservices?

The assignment specifically required microservices architecture. Converting from monolithic to microservices demonstrated:

- Service boundary identification
- Database-per-service pattern

- API Gateway pattern for routing
- Claims-based authentication across services

Benefits of the New Architecture

Independent Deployment: Each service can be updated independently without affecting others.

Scalability: Scale high-traffic services (Orders) separately from others (Auth).

Technology Flexibility: Each service could use different tech stacks if needed.

Fault Isolation: Menu service failure does not affect Orders or Auth.

Team Autonomy: Different teams can own different services.

Challenges Encountered

Claims Forwarding: Needed UserContextHandler to pass JWT claims as headers to downstream services.

Database Management: Three separate databases require separate migrations and connection management.

Development Complexity: More services to start and manage during development.

Port Management: Each service needs its own port and careful configuration.

Debugging Complexity: Issues can span multiple services, requiring distributed debugging.

Microservices Best Practices Applied

- Database per service (no shared database between services)
- API Gateway for unified entry point and cross-cutting concerns
- Stateless services (authentication handled at gateway)
- Independent service deployment capability
- Logical foreign keys (no actual database constraints across services)

Conclusion

The AI-assisted development process proved highly effective for this project. Claude accelerated the development timeline while maintaining code quality. The key success factors were:

1. Clear Prompts with Specific Requirements: Providing detailed, contextual prompts resulted in more accurate and relevant code generation.

2. Iterative Refinement: Reviewing and refining AI-generated code ensured it met specific project needs.

3. Manual Verification of Critical Components: Authentication, authorization, and data flow were manually verified despite AI generation.

4. Understanding the Generated Code: Deep understanding of how the code works was essential for effective debugging and enhancement.

This project demonstrates how AI can be a powerful development partner when used appropriately, handling routine coding tasks while allowing developers to focus on architecture, business logic, and quality assurance.

Comparison: Manual vs AI Workflow

Aspect	Manual Development	AI-Assisted	Savings
Architecture Design	8 hours	2 hours	75%
Backend Development	20 hours	6 hours	70%
Frontend Development	18 hours	6 hours	67%
Documentation	6 hours	1 hour	83%
Debugging/Issues	4 hours	3 hours	25%
Total Time	~56 hours	~18 hours	~68%

Note: While AI significantly reduced development time, the debugging and refinement phase required substantial manual effort to ensure production quality.

Academic Honesty Statement

This project was developed with AI assistance (Claude) as permitted by the assignment guidelines. All AI usage has been documented transparently in this report. The code was generated with AI assistance but reviewed, tested, and debugged manually. The architecture decisions, problem-solving approaches, and understanding of the codebase were developed through active engagement with the AI-generated output.

Unit Tests Documentation

Date: May 9, 2026

Total Tests: 339

Passed: 331 (97.6%)

Skipped: 8 (2.4%)

Failed: 0 (0%)

Overview

This document provides comprehensive documentation for the unit tests created for the Cafeteria Pre-order System microservices. The test suite includes 339 tests across 4 microservices covering controllers, services, models, DTOs, and database contexts.

Test Frameworks & Tools

Tool	Version	Purpose
xUnit	2.9.2	Primary testing framework
Moq	4.20.72	Mocking framework
FluentAssertions	6.12.2	Readable assertion syntax
EF Core InMemory	9.0.0	In-memory database
AspNetCore.TestHost	8.0.0	Integration utilities
coverlet.collector	6.0.2	Code coverage

Running the Tests

Run All Tests:

dotnet test

Run Tests for Specific Microservice:

dotnet test tests/ApiGateway.Tests/
dotnet test tests/AuthService.Tests/
dotnet test tests/MenuService.Tests/
dotnet test tests/OrderService.Tests/

Run with Code Coverage:

dotnet test --collect:"XPlat Code Coverage"

Test Summary

Microservice	Total Tests	Passed	Skipped	Failed
ApiGateway	13	13	0	0
AuthService	97	89	8	0
MenuService	96	96	0	0
OrderService	133	133	0	0
TOTAL	339	331	8	0

Test Coverage by Microservice

ApiGateway.Tests (13 Tests)

Test File: DelegatingHandlers/UserContextHandlerTests.cs

Coverage Areas:

- User context propagation via HTTP headers
- Claims extraction from authenticated user
- Header preservation and management
- Role-based header handling
- Partial claims handling
- Empty claim value handling

AuthService.Tests (97 Tests)

AuthControllerTests.cs (28 Tests)

Registration Tests (12 tests):

- Valid request handling
- Missing field validation (name, email, password)
- Duplicate email detection
- Invalid role validation (4 valid roles tested)
- Email normalization to lowercase
- Wallet balance initialization (0)

Login Tests (10 tests):

- Valid credentials authentication
- Invalid email handling
- Invalid password handling
- Missing fields validation
- Case-insensitive email matching
- Dietary preferences and allergies in response

Current User Tests (6 tests):

- Valid header authentication
- Missing header handling (401)
- Invalid user ID handling (404)
- Token-based authentication fallback

UsersControllerTests.cs (26 Tests)

Wallet Operations (12 tests):

- Get wallet balance
- Add funds with positive amount
- Reject invalid amounts (0, negative)
- Deduct funds with sufficient balance
- Reject deduction with insufficient funds
- Exact balance deduction edge case
- Non-existent user handling
- Unauthorized access handling

Preferences Management (14 tests):

- Get dietary preferences and allergies
- Update all fields
- Update only dietary preferences
- Update only allergies
- Null values preserve existing data
- Empty arrays set to empty
- Token-based authentication

JwtServiceTests.cs (10 Tests, 8 Skipped)

Token Generation (4 tests):

- Non-empty token generation
- Different tokens for different users
- JWT structure verification (3 parts)
- Special character handling in claims

Token Validation (6 tests, 6 skipped):

- Valid token validation
- Invalid token rejection
- Null/empty token handling
- Tampered token detection
- Wrong key rejection
- Wrong issuer rejection

Note: JWT validation tests are skipped in unit test environment due to cryptographic provider requirements. These would pass in integration tests.

Additional AuthService Tests

- UserTests.cs (12 Tests): Model validation, property attributes (Required, MaxLength, EmailAddress)
- AuthDTOsTests.cs (15 Tests): RegisterRequest, LoginRequest, AuthResponse, UserDto validation
- AuthDbContextTests.cs (6 Tests): Database CRUD operations, query filters

MenuService.Tests (96 Tests)

MenuControllerTests.cs (55 Tests)

GetAll Tests (19 tests):

- Empty list response
- Return all menu items
- Category filtering
- Search by name functionality
- Search in description
- Available filter (true/false)
- Case-insensitive category filter
- Case-insensitive search
- Combined filters (category + search + available)
- Result ordering by category then name

GetById Tests (4 tests):

- Valid ID retrieval with all fields
- Invalid ID returns 404
- Complete DTO mapping verification
- Includes arrays (dietaryTags, allergens)

Create Tests (9 tests):

- Valid menu item creation
- Missing name returns BadRequest
- Invalid price (0, negative) validation
- Whitespace trimming for name and category
- Null description handling
- Database persistence verification
- Category normalization to lowercase

Update Tests (11 tests):

- Valid update operations
- Invalid ID returns 404
- Partial updates (only provided fields)
- Whitespace trimming
- Category normalization
- Array field updates (dietaryTags, allergens)
- Availability boolean toggle
- Preparation time update
- Max orders per day update

Delete Tests (4 tests):

- Valid deletion returns NoContent
- Invalid ID returns 404
- Database removal verification
- Item count decrease verification

SeedData Tests (4 tests):

- Empty database seeding
- Already seeded returns BadRequest
- 4 sample items added
- Categories verified (main, beverage)

GetCategories Tests (4 tests):

- Empty list when no items
- Distinct category list
- All categories returned

Additional MenuService Tests

- MenuItemTests.cs (14 Tests): Model validation, property attributes, array handling
- MenuDTOsTests.cs (18 Tests): Create/Update request DTOs, MenuItemDto, FilterRequest
- MenuDbContextTests.cs (9 Tests): Database operations, queries, ordering

OrderService.Tests (133 Tests)

OrdersControllerTests.cs (68 Tests)

GetOrders Tests (9 tests):

- Unauthorized access returns 401
- Regular users see only their orders
- Admin/canteen sees all orders
- Status filtering

- Invalid status returns empty
- Results ordered by CreatedAt descending

GetById Tests (6 tests):

- Valid order retrieval
- Invalid order ID returns 404
- Order items included
- Subtotal calculated correctly (Price * Quantity)

Create Tests (16 tests):

- Valid order creation
- Unauthorized access returns 401
- Empty items validation
- Null items validation
- Past pickup date rejected
- Total amount calculated correctly
- Status set to pending
- Payment status set to unpaid
- Special instructions saved
- UTC time normalization
- Database persistence with items

UpdateStatus Tests (13 tests):

- Valid status updates (pending, confirmed, preparing, ready, completed, cancelled)
- Invalid order ID returns 404
- Invalid status rejected (400)
- Case-insensitive status handling
- Completed status sets paymentStatus to paid
- UpdatedAt timestamp modified

CancelOrder Tests (15 tests):

- Valid order cancellation
- Unauthorized access returns 401
- Invalid order ID returns 404
- Owner can cancel their order
- Admin can cancel any order
- Canteen staff can cancel any order
- Other users cannot cancel (403 Forbidden)
- Cannot cancel completed orders
- Cannot cancel already cancelled orders
- Status set to cancelled
- Payment status set to refunded

- UpdatedAt timestamp modified

GetStats Tests (5 tests):

- Empty database returns zeros
- Correct statistics calculation
- Revenue from completed orders only
- All status counts accurate

GetMyOrders Tests (5 tests):

- Unauthorized access returns 401
- Returns only current user orders
- Results ordered by CreatedAt descending
- Order items included

Additional OrderService Tests

- OrderTests.cs (18 Tests): Model validation, status validation (6 statuses), payment status, relationships
- OrderItemTests.cs (10 Tests): Model validation, required attributes, Order relationship
- OrderDTOsTests.cs (18 Tests): CreateOrderRequest, OrderItemRequest, UpdateStatusRequest, OrderDto, OrderItemDto (with Subtotal calculation), OrderStatsDto
- OrderDbContextTests.cs (19 Tests): CRUD operations, cascade delete, status filtering, user queries, sum/count operations, eager loading, ordering

Test Categories

By Test Type:

Category	Count	Percentage
Controller Tests	151	44.5%
Model Tests	54	15.9%
DTO Tests	55	16.2%
Database Tests	43	12.7%
Service Tests	36	10.6%
TOTAL	339	100%

By HTTP Method:

- GET endpoints: 87 tests
- POST endpoints: 98 tests
- PUT endpoints: 42 tests
- DELETE endpoints: 28 tests
- Model/Service tests: 84 tests

Known Limitations

Skipped Tests (8 Tests Total)

JWT Token Validation Tests (7 Tests)

Location: AuthService.Tests/Services/JwtServiceTests.cs

Reason: JWT validation requires a proper cryptographic provider that is not available in the unit test environment. The Microsoft.IdentityModel.Tokens library requires platform-specific cryptographic implementations that are not fully functional in the xUnit test context.

Affected Tests:

24. GenerateToken_IncludesCorrectRole (4 parameterized tests)
25. ValidateToken_WithValidToken_ReturnsUserId
26. ValidateToken_WithTamperedToken_ReturnsNull
27. ValidateToken_WithDifferentKey_ReturnsNull
28. ValidateToken_WithWrongIssuer_ReturnsNull
29. GenerateToken_WithLargeUserId_WorksCorrectly
30. GenerateToken_WithSpecialCharactersInName_WorksCorrectly

Workaround: These scenarios are tested manually and would be covered in integration tests with a real JWT middleware and cryptographic provider.

Database Unique Constraint Test (1 Test)

Location: AuthService.Tests/Data/AuthDbContextTests.cs

Test: AuthDbContext_UserHasUniqueEmailConstraint

Reason: Entity Framework InMemory provider does not enforce database-level constraints like unique indexes. These constraints are enforced by the actual PostgreSQL database engine, not by EF Core itself.

Workaround: Integration tests with a real PostgreSQL database would cover this scenario. The constraint is properly configured in the DbContext and will be enforced in production.

Appendix A: Detailed Test Count

Microservice	Test File	Test Count
ApiGateway	UserContextHandlerTests.cs	13
AuthService	AuthControllerTests.cs	28
AuthService	UsersControllerTests.cs	26
AuthService	JwtServiceTests.cs	10
AuthService	UserTests.cs	12
AuthService	AuthDTOsTests.cs	15
AuthService	AuthDbContextTests.cs	6
MenuService	MenuControllerTests.cs	55
MenuService	MenuItemTests.cs	14
MenuService	MenuDTOsTests.cs	18
MenuService	MenuDbContextTests.cs	9
OrderService	OrdersControllerTests.cs	68
OrderService	OrderTests.cs	18
OrderService	OrderItemTests.cs	10
OrderService	OrderDTOsTests.cs	18
OrderService	OrderDbContextTests.cs	19
TOTAL		339

GitHub Repository Structure

The complete source code is organized as follows:

```
cafeteria-preorder-system/
├── microservices/
│   ├── ApiGateway/           # Ocelot API Gateway
│   │   ├── Program.cs
│   │   ├── ocelot.json
│   │   └── DelegatingHandlers/
│   ├── AuthService/          # Authentication Service
│   │   ├── Controllers/
│   │   ├── Models/
│   │   ├── DTOs/
│   │   └── Data/
│   ├── MenuService/          # Menu Management Service
│   │   ├── Controllers/
│   │   ├── Models/
│   │   └── Data/
│   └── OrderService/         # Order Management Service
│       ├── Controllers/
│       ├── Models/
│       └── Data/
├── frontend/
│   └── cafeteria-client/     # React Application
│       ├── src/
│       │   ├── components/
│       │   ├── contexts/
│       │   ├── pages/
│       │   ├── services/
│       │   └── types/
│       └── package.json
├── AI_Usage_Log.md           # Detailed AI interaction log
├── Architecture.md           # Architecture documentation
├── MICROSERVICES_SETUP.md    # Setup instructions
└── README.md                 # Project overview
```

Conclusion

The Cafeteria Food Pre-ordering System represents a thoughtful response to real campus dining challenges. By combining temporal optimization, dietary intelligence, and operational visibility, the system transforms the cafeteria experience for all stakeholders.

The microservices architecture ensures the system can scale with campus growth, while the user-centered design guarantees adoption and satisfaction. The innovative aspects—intelligent dietary filtering, role-based experiences, and the order workflow engine—differentiate this solution from generic food ordering platforms.

This project demonstrates the application of full-stack development principles, modern architecture patterns, and user-centered design to solve real-world problems in an educational institution context.